



Library Management System

Complete Project Guide & Simple Explanation for Students & Evaluators

1. What is this Project? (Simple Overview)

This project is a **Desktop Application** built in **Java** to help a library manage its day-to-day operations easily. It provides a visual graphical interface (GUI) where librarians can perform all key tasks:

- **Manage Books & E-Books:** Add new items, update details, search by title/author/category, and delete old items.
- **Manage Members:** Register Students and Faculty members with specific loan rules (students can borrow 3 books for 14 days; faculty can borrow 10 books for 30 days with a 50% discount on overdue fines).
- **Issue & Return Books:** Issue books to users, track return due dates, and calculate overdue fines automatically when books are returned late.
- **Background Tasks (Multithreading):** Calculate overdue fines for all users in the background and export library reports to CSV files without slowing down the application window.

2. What Should You Do to Run & Test the Project?

Follow these 3 easy steps to run the application on your computer:

Step 1: Open the Project Folder

Go to your Desktop folder: `C:\Users\abhinav\OneDrive\Desktop\Library Management System`

Step 2: Compile the Code

Double-click the file named `build.bat`. This compiles all Java files into runnable class files.

Step 3: Launch the Application

Double-click the file named `run.bat`. The main window will pop up automatically on your screen!

3. Simple Explanation of Key Project Modules

A. Object-Oriented Programming (OOP)

The system uses core OOP concepts to structure the code neatly:

- **Abstraction & Interfaces (`Borrowable`, `Searchable`):** Defines standard actions (like issuing, returning, and searching) that entities must follow.
- **Inheritance (Base & Child Classes):** `LibraryItem` is the parent class for `Book` and `EBook`. `User` is the parent class for `StudentUser` and `FacultyUser`.
- **Polymorphism (Dynamic Rules):** `calculateOverdueFine()` charges physical books \$1.50/day and digital e-books \$0.50/day. Faculty members get a 50% discount on overdue charges.
- **Custom Exception Handling:** Prevents crashes when errors occur (e.g., trying to borrow an out-of-stock book throws a custom `BookNotAvailableException`).

B. Database Connectivity (JDBC & SQLite)

All data is saved in a local database file named `library.db` using **JDBC (Java Database Connectivity)** so your books and user profiles remain saved even when you close and re-open the application.

C. Multithreading (Background Operations)

The application uses separate background worker threads so heavy tasks run smoothly in the background:

- **Fine Sweep Task (`Runnable`):** Audits all active loans and updates overdue charges without locking the screen.
- **Catalog Export Task (`Callable`):** Exports book inventory data to a CSV spreadsheet file asynchronously.

4. Project Marking Rubric Summary

Rubric Requirement	Implementation in Code	Status
OOP Implementation	Interfaces (Borrowable, Searchable), Inheritance (Book, EBook, User), Polymorphic fines, Custom Exceptions	Complete
Collections & Generics	Generic DAO (GenericDAO<T, ID>), ArrayLists, and HashMap memory caches	Complete
Multithreading & Synchronization	FineCalculationTask (Runnable), BackupTask (Callable), ReentrantLock synchronization	Complete
Database Operations (DAO)	Clean Data Access Object pattern: BookDAO, UserDAO, TransactionDAO	Complete
Database Connectivity (JDBC)	SQLite JDBC connection manager with PreparedStatement & ResultSet mapping	Complete

5. What to Submit to Your Evaluator / Teacher

When submitting your project, provide the following 2 items:

1. GitHub Repository URL:

<https://github.com/chandraabhinav68-pixel/Library-Management-System>

2. Presentation & Guide PDF Files:

- Library_Management_System_Presentation.pdf (Visual slides & architecture)
- Project_Guide_and_Explanation.pdf (This simple explanation guide)